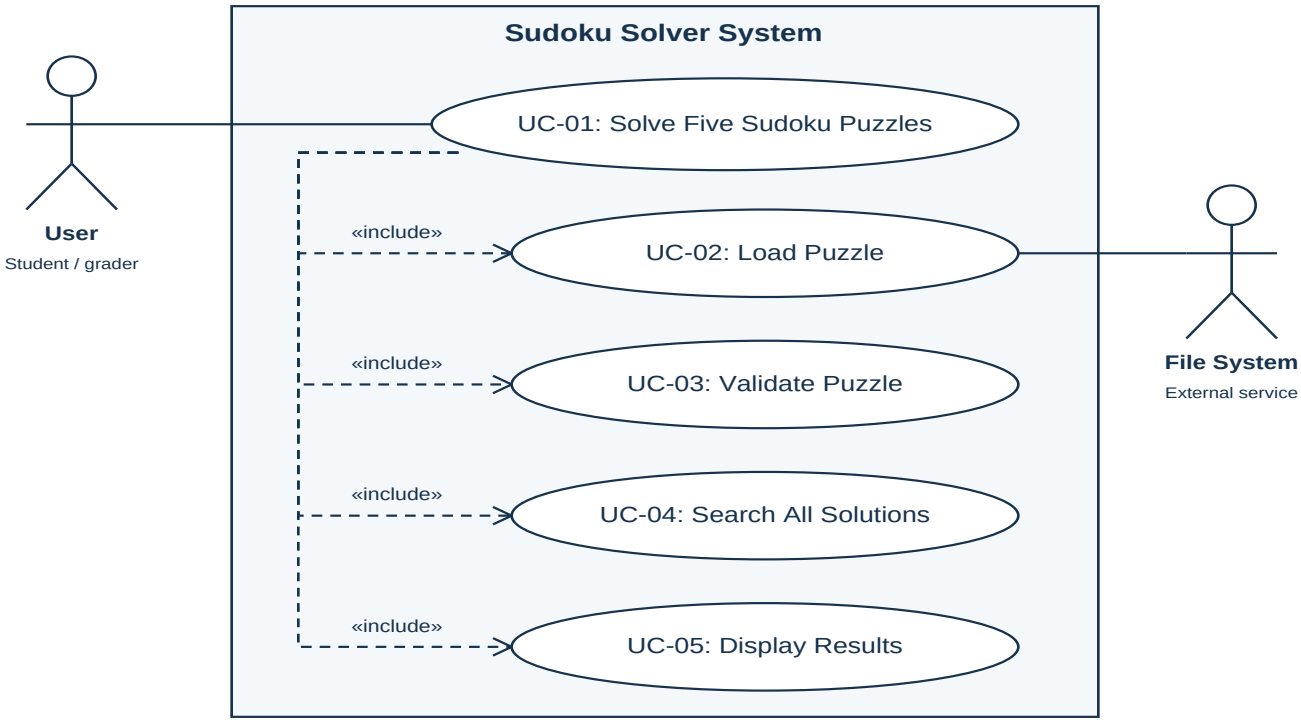
Sudoku Solver: UML Use Case

EECS 348 - Assignment 4 | prepared with assistance from GPT-6.1 Sol

a. Identify / discover a use case

UC-01: Solve Five Sudoku Puzzles. The user runs the program to obtain every valid completion or a no-solution result for each assigned puzzle. The File System is a supporting actor that supplies puzzle-file contents or file-access failures.



Solid lines show actor associations. Dashed <<include>> arrows point from UC-01 to its required sub-use cases in the normal flow. File and validation exceptions interrupt that flow before search. The diagram shows responsibilities, not execution order; the flows below specify order.

b. Brief description

The user runs the Sudoku solver once. The system automatically processes puzzle1.txt through puzzle5.txt in order. It loads each puzzle through the File System, validates its format and clues, displays the original grid, searches for every valid completion, and displays all solutions or "No solution found." A file-specific error is reported when loading fails, and processing continues with the next file.

c. Step-by-step flow (outline)

1. User starts the program in an environment containing the five puzzle files.

2. System selects the next filename, beginning with puzzle1.txt, and displays that filename.

3. System opens the file through the File System and reads its puzzle cells (UC-02).

4. System checks that the input contains exactly 81 cells using digits 1-9 or underscores; whitespace is ignored (UC-03). If reading or format validation fails, it reports the error and moves to the next file.

5. System displays the original puzzle, with underscores representing blank cells (UC-05).

6. System checks the starting clues for row, column, and 3 x 3 block conflicts (UC-03). If they conflict, it prints "No solution found." and moves to the next file.

7. System searches for all valid completions while preserving the starting clues (UC-04).

8. System prints each completed solution as it is found and continues searching for additional solutions (UC-05).

9. After the search ends, system prints the solution count, or "No solution found." if none were found (UC-05).

10. System repeats the process through puzzle5.txt, then terminates. User reviews the terminal output.

d. Detailed description

Use case / scope	UC-01: Solve Five Sudoku Puzzles / Sudoku Solver System
Actors	Primary: User (student or grader). Supporting: File System, which provides access to the input files.
Level / trigger	User goal; one execution processes five files. Trigger: user launches the compiled program.
Preconditions	The executable is available and can be run. The user places the intended puzzle files in the working directory. Missing or malformed files are handled as exceptions.
Inputs	puzzle1.txt through puzzle5.txt; each intended to contain 81 cell characters.
Outputs	Terminal text: filenames, original readable puzzles, all solutions or a no-solution message, solution totals when applicable, and file-specific errors.

d. Detailed description (continued)

Main success scenario

Step	Actor action / system response
1	User launches the program once; no interactive filename or digit entry is required.
2	System begins the five-file batch. For the current file, it prints a filename heading. UC-02 requests file access from the File System and reads cell characters.
3	UC-03 verifies exactly 81 allowed cell characters, ignoring whitespace. UC-05 displays the original grid; UC-03 then confirms that the fixed clues do not conflict.
4	UC-04 explores candidate completions using recursive depth-first search with backtracking. Each placed digit must satisfy its row, column, and 3 x 3 block constraints. Original clues remain unchanged.
5	Whenever a complete valid grid is reached, UC-05 prints a numbered solution. UC-04 resumes the search rather than stopping at the first solution.
6	After all candidate paths have been explored, UC-05 prints the total number of solutions found for that puzzle.
7	System repeats steps 2-6 for each remaining file through puzzle5.txt, then exits. User receives the results in the terminal.

Alternative and exception flows

2a. File unavailable: If a file is missing or unreadable, system prints an "Unable to read" message identifying the filename and cause, skips that file, and continues with the next file.

3a. Invalid format: If a file contains an invalid non-whitespace character, fewer than 81 cells, or more than 81 cells, system reports the specific reason. It does not solve the partial input and continues with the next file.

3b. Conflicting clues: If starting digits violate a row, column, or block constraint, system prints "No solution found." without searching and continues with the next file.

4a. No completion: If starting clues are consistent but exhaustive search finds no valid completion, system prints "No solution found." and continues with the next file.

5a. Multiple / complete: For multiple solutions, system prints every distinct completed grid. An already complete valid input is printed as one solution; an already complete invalid input follows 3b.

Postconditions and rules

Every file has been attempted independently. Each validly loaded puzzle has all its solutions printed or a no-solution result; a failed load has a file-specific error. Input files and original clues are not modified. Each solution contains digits 1-9 exactly once in every row, column, and 3 x 3 block. Search continues after each solution so that no valid completion is intentionally omitted.